

ساخت نخستین ربات تلگرام

آموزش عملی و گام به گام با TeleBot

از یک پوشه خالی شروع کنید و با رباتی تمام کنید که خوشامد می گوید،
دستورهایش را توضیح می دهد و پیام های شما را تکرار می کند.

Alireza Khajevandi

وبسایت • GitHub • LinkedIn

YouTube • تلگرام • Instagram

روش استفاده از این آموزش

این راهنما کاملاً عملی است. فصل‌ها را به‌ترتیب انجام دهید و در هر ایستگاه بررسی توقف کنید. دستورها آماده‌ی کپی هستند؛ با این حال توضیح کوتاه کنار هر دستور را بخوانید تا بدانید چه چیزی تغییر می‌کند. به تجربه‌ی قبلی ساخت ربات نیاز ندارید.

نسخه‌های فارسی و انگلیسی، پیاده‌سازی، دستورها، نتیجه‌های مورد انتظار و نکته‌های امنیتی یکسانی دارند. فقط زبان و جهت صفحه‌های آن‌ها متفاوت است.

نکته مهم

توکن ربات تلگرام را هرگز با دیگران به اشتراک نگذارید. در نمونه‌ها فقط از مقدار جایگزین استفاده شده است. فایل خروجی PDF در گیت‌هاب قرار گرفته است. ولی فایل‌های ساخت لاک در گیت ذخیره نمی‌شوند؛ و هر بار اگر شما آن‌ها را بسازید مجدد روی سیستم شما تشکیل خواهند شد.

فهرست مطالب

۱	آشنایی با رباتی که می‌سازیم	۱
۱	۱.۱ چهار بخش اصلی	۱
۱	۲.۱ ساختاری کوچک و خوانا	۱
۳	۲ آماده‌سازی پایتون و پروژه	۳
۳	۱.۲ بازکردن پوشهٔ کد	۳
۳	۲.۲ ساخت محیط جداگانه	۳
۵	۳ ساخت ربات با بات‌فادر	۵
۵	۱.۳ درخواست ربات از بات‌فادر	۵
۵	۲.۳ تنظیم توکن برای یک نشست ترمینال	۵
۷	۴ ساخت و درک ربات	۷
۹	۱.۴ گام اول: واردکردن ابزارها	۹
۹	۲.۴ گام دوم: پاسخ‌هایی که آسان آزموده می‌شوند	۹
۹	۳.۴ گام سوم: ثبت پاسخ‌دهنده‌ها	۹
۹	۴.۴ گام چهارم: آغاز دریافت پیام	۹
۱۰	۵ اجرا و بررسی نتیجه	۱۰
۱۰	۱.۵ بررسی دستی در چهار بخش	۱۰
۱۰	۲.۵ اجرای آزمون‌های آفلاین	۱۰
۱۲	۶ رفع مشکل‌های رایج و ادامهٔ مسیر	۱۲
۱۲	۱.۶ راهنمای سریع مشکل‌ها	۱۲
۱۲	۲.۶ ساخت این کتاب در تک‌استودیو	۱۲
۱۳	۳.۶ گام‌های کوچک بعدی	۱۳

فصل ۱

آشنایی با رباتی که می‌سازیم

هدف فصل

نتیجه نهایی، ابزارهای اندک مورد نیاز و ساختار پروژه را بشناسید.

هدف ما آگاهانه کوچک انتخاب شده است: ساخت ربات تلگرامی که تمام کردن و فهمیدن آن آسان باشد. این ربات کارهای زیر را انجام می‌دهد:

- وقتی کاربر دستور `/start` را می‌فرستد، به او خوشامد می‌گوید؛
- با دریافت دستور `/help` روش کار خود را توضیح می‌دهد؛
- به متن عادی با عبارت `You said:` و سپس همان متن پاسخ می‌دهد؛
- پس از دریافت عکس، پیام صوتی یا محتوای مشابه، از کاربر متن می‌خواهد.

برای نمونه، وقتی علیرضا پیام زیر را می‌فرستد، گفت‌وگو چنین خواهد بود:

انتظار مورد نتیجه

Alireza: My first bot

Bot: You said: My first bot

۱.۱ چهار بخش اصلی

تلگرام گفت‌وگو را نمایش می‌دهد. BotFather حساب ربات را می‌سازد و توکن محرمانه آن را تحویل می‌دهد. پایتون دستورهای ما را اجرا می‌کند. بسته `pyTelegramBotAPI` ماژول `telebot` را فراهم می‌کند تا این دستورها به تلگرام متصل شوند. در این آموزش از روش `polling` استفاده می‌کنیم؛ یعنی تا وقتی برنامه باز است، مرتب از تلگرام می‌پرسد آیا پیام تازه‌ای رسیده است یا نه.

۲.۱ ساختاری کوچک و خوانا

```
telbot-simple-bot/  
|-- code/  
|   |-- bot.py  
|   |-- requirements.txt  
|   |-- test_bot.py
```

```
| `-- README.md  
|-- documentation/  
| |-- English/  
| `-- Persian/  
|-- LINKEDIN_POST.md  
`-- README.md
```

پس از نصب وابستگی، فقط فایل code/bot.py برای اجرای ربات لازم است. فایل‌های دیگر برای آموزش، آزمون یا مستندسازی هستند.

ایستگاه بررسی

اکنون دقیقاً می‌دانید ربات نهایی چه می‌کند. هنوز هیچ توکن، بسته یا حسابی تغییر نکرده است.

فصل ۲

آماده‌سازی پایتون و پروژه

هدف فصل

یک محیط جداگانه پایتون بسازید و تنها بسته لازم را نصب کنید.

پایتون نسخه ۳/۱۰ یا جدیدتر را نصب کنید. نسخه را در ترمینال، پاورشل یا ترمینال داخل ویرایشگر بررسی کنید:

```
python --version
```

در بعضی سیستم‌های مک یا لینوکس باید از دستور python3 استفاده کنید. نتیجه‌ای مانند Python 3.12.4 مناسب است.

۱.۲ بازکردن پوشه کد

مخزن را دریافت کنید و وارد پوشه کد شوید:

```
git clone https://github.com/alirezaaies/telbot-simple-bot.git
cd telbot-simple-bot/code
```

اگر فایل فشرده را دانلود کرده‌اید، آن را استخراج کنید و ترمینال را در پوشه code باز کنید.

۲.۲ ساخت محیط جداگانه

محیط مجازی، بسته این آموزش را از پروژه‌های دیگر پایتون جدا نگه می‌دارد.
در مک یا لینوکس:

```
python3 -m venv .venv
source .venv/bin/activate
```

در پاورشل ویندوز:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

ترمینال معمولاً عبارت (.venv) را پیش از نشانگر خود نمایش می‌دهد. سپس وابستگی را نصب کنید:

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

فایل نیازمندی‌ها فقط شامل بسته pyTelegramBotAPI است. نام آن هنگام واردکردن در پایتون telebot است؛ این تفاوت طبیعی است.

ایستگاه بررسی

دستور زیر را اجرا کنید. اگر عبارت telebot is ready نمایش داده شد، نصب کامل است.

```
python -c "import telebot; print('telebot is ready')"
```


فصل ۳

ساخت ربات با بات فادر

هدف فصل

حساب ربات تلگرام را بسازید و توکن آن را به شکل امن در اختیار پایتون بگذارید.

۱.۳ درخواست ربات از بات فادر

۱. در تلگرام حساب تأییدشده @BotFather را جستجو کنید.
۲. گفتگو را آغاز کنید و دستور newbot را بفرستید.
۳. یک نام نمایشی وارد کنید؛ برای نمونه Alireza Tutorial Bot مناسب است.
۴. نام کاربری یکتایی وارد کنید که به bot پایان برسد؛ برای نمونه alireza_tutorial_123_bot را امتحان کنید. اگر قبلاً گرفته شده است، نام دیگری بسازید.
۵. بات فادر یک توکن می‌دهد. با آن مانند رمز عبور رفتار کنید.

نکته مهم

توکن واقعی را در کد، تصویر، پیام یا گیت قرار ندهید. اگر توکن فاش شد، دستور revoke / را برای بات فادر بفرستید و توکن تازه‌ای بسازید.

۲.۳ تنظیم توکن برای یک نشست ترمینال

مقدار نمونه را با توکنی که از بات فادر گرفته‌اید جایگزین کنید.
در مک یا لینوکس:

```
export BOT_TOKEN="paste_your_token_here"
```

در پاورشل ویندوز:

```
$env:BOT_TOKEN="paste_your_token_here"
```

با بسته‌شدن آن ترمینال، متغیر حذف می‌شود. این روش برای نخستین پروژه ساده و امن است. در هر نشست تازه باید آن را دوباره تنظیم کنید.

ایستگاه بررسی

فقط وجود متغیر را بررسی کنید و مقدار محرمانه آن را نمایش ندهید.

```
python -c "import os; print('token set' if os.getenv('BOT_TOKEN') else 'missing')"
```

نتیجه مورد انتظار عبارت token set است.

فصل ۴

ساخت و درک ربات

هدف فصل

توکن را به تله‌بات متصل کنید و بفهمید هر پیام چگونه پاسخ خود را می‌گیرد.

فایل کامل از قبل در مخزن قرار دارد. خواندن آن از بالا به پایین، سریع‌ترین راه برای ارتباط دادن هر مفهوم با کد عملی است.

Listing :1.۴ The complete code in code/bot.py

```
1 """Run a small Telegram bot with the pyTelegramBotAPI (telebot) library.
2
3 The bot answers /start and /help, repeats normal text messages, and
4 explains
5 what to do when it receives another content type. The Telegram token is
6 read
7 from the BOT_TOKEN environment variable so that no secret is stored in code
8 .
9
10 Author: Alireza Khajehvandi
11 Project: https://github.com/alirezaaies/telbot-simple-bot
12 """
13
14 from __future__ import annotations
15
16 import os
17
18 import telebot
19 from telebot import types
20
21 def start_reply(first_name: str | None) -> str:
22     """Return the welcome message used by the /start command."""
23     name = first_name or "there"
24     return (
25         f"Hello, {name}!\n\n"
26         "I am your first Telegram bot.\n"
27         "Send me any text and I will repeat it.\n\n"
28         "Commands:\n"
29         "/start - show the welcome message\n"
30         "/help - show usage help"
31     )
```

```

131 def help_reply() -> str:
132     """Return a short, beginner-friendly help message."""
133     return (
134         "How to use this bot:\n"
135         "1. Send any text message.\n"
136         "2. The bot replies with the same text.\n"
137         "3. Send /start whenever you want to see the welcome message again."
138     )
139
140
141 def echo_reply(text: str) -> str:
142     """Build the reply for a normal text message."""
143     return f"You said: {text}"
144
145
146 def create_bot(token: str) -> telebot.TeleBot:
147     """Create the bot and register all message handlers."""
148     if not token.strip():
149         raise ValueError("The Telegram bot token cannot be empty.")
150
151     bot = telebot.TeleBot(token)
152
153     @bot.message_handler(commands=["start"])
154     def handle_start(message: types.Message) -> None:
155         """Welcome the user and list the available commands."""
156         bot.reply_to(message, start_reply(message.from_user.first_name))
157
158     @bot.message_handler(commands=["help"])
159     def handle_help(message: types.Message) -> None:
160         """Explain how the bot behaves."""
161         bot.reply_to(message, help_reply())
162
163     @bot.message_handler(content_types=["text"])
164     def handle_text(message: types.Message) -> None:
165         """Repeat every ordinary text message."""
166         bot.reply_to(message, echo_reply(message.text or ""))
167
168     @bot.message_handler(
169         content_types=["audio", "document", "photo", "sticker", "video", "voice"]
170     )
171     def handle_unsupported_content(message: types.Message) -> None:
172         """Guide the user when this tutorial bot receives non-text content."""
173         bot.reply_to(message, "Please send a text message so I can repeat it.")
174
175     return bot
176
177
178 def main() -> None:
179     """Read configuration and keep the bot connected to Telegram."""
180     token = os.getenv("BOT_TOKEN")
181     if not token:
182         raise SystemExit(
183             "BOT_TOKEN is missing. Export it in your terminal, then run bot.py again."
184         )

```

```

۸۵ )
۸۶
۸۷ bot = create_bot(token)
۸۸ print("Bot is running. Press Ctrl+C to stop it.")
۸۹ bot.infinity_polling(skip_pending=True)
۹۰
۹۱
۹۲ if __name__ == "__main__":
۹۳     main()

```

۱.۴ گام اول: وارد کردن ابزارها

ماژول os متغیر محیطی را می‌خواند. telebot برنامه‌ای می‌سازد که با تلگرام صحبت می‌کند و types نوع پیام‌ها را برای خواننده و ویرایشگر مشخص می‌کند. این نوع‌ها رفتار ربات را تغییر نمی‌دهند.

۲.۴ گام دوم: پاسخ‌هایی که آسان آزموده می‌شوند

تابع‌های start_reply و help_reply و echo_reply فقط متن می‌سازند و به اینترنت نیاز ندارند. جدا کردن آن‌ها گفت‌وگوی مورد انتظار را روشن می‌کند و به آزمون‌های آفلاین اجازه می‌دهد نتیجه را بررسی کنند.

۳.۴ گام سوم: ثبت پاسخ‌دهنده‌ها

پاسخ‌دهنده یا handler فقط تابعی است که برای یک نوع پیام ورودی انتخاب می‌شود. دستورهای بالای تابع‌ها این انتخاب را مشخص می‌کنند:

- عبارت `commands=["start"]` دستور /start را مدیریت می‌کند؛
- عبارت `commands=["help"]` دستور /help را مدیریت می‌کند؛
- عبارت `content_types=["text"]` متن‌های دیگر را مدیریت می‌کند؛
- آخرین پاسخ‌دهنده، پیام‌های رایج غیرمتنی را پوشش می‌دهد.

ترتیب مهم است: پاسخ‌دهنده‌های دستور پیش از پاسخ‌دهنده عمومی متن قرار گرفته‌اند؛ پس هر دستور پاسخ ویژه خودش را می‌گیرد.

۴.۴ گام چهارم: آغاز دریافت پیام

تابع main در صورت نبودن BOT_TOKEN با خطایی خوانا متوقف می‌شود. اگر توکن وجود داشته باشد، تابع infinity_polling تا زمان فشردن Ctrl+C به دریافت پیام ادامه می‌دهد. گزینه skip_pending=True پیام‌های قدیمی جمع‌شده در زمان خاموش بودن این ربات آموزشی را نادیده می‌گیرد.

ایستگاه بررسی

اکنون می‌توانید پاسخ‌دهنده دقیق دستورهای /start و /help و همچنین متن عادی و عکس را در کد نشان دهید.

فصل ۵

اجرا و بررسی نتیجه

هدف فصل

ربات را روشن کنید، یک گفت‌وگوی واقعی در تلگرام انجام دهید و آزمون‌های آفلاین را اجرا کنید.

مطمئن شوید محیط مجازی فعال و متغیر BOT_TOKEN تنظیم شده است؛ سپس اجرا کنید:

```
python bot.py
```

انتظار مورد نتیجه

Bot is running. Press Ctrl+C to stop it.

ترمینال را باز نگه دارید. در تلگرام پیوند داده‌شده توسط بات‌فادر را باز کنید یا نام کاربری ساخته‌شده را جست‌وجو کنید. دکمه شروع را بزنید.

۱.۵ بررسی دستی در چهار بخش

۱. دستور /start را بفرستید. پاسخ با نام کوچک شما در تلگرام آغاز می‌شود و هر دو دستور را نشان می‌دهد.

۲. دستور /help را بفرستید. پاسخ سه گام کوتاه استفاده را نمایش می‌دهد.

۳. پیام My first bot را بفرستید. پاسخ باید دقیقاً عبارت زیر باشد: You said: My first bot

۴. یک عکس بفرستید. پاسخ باید دقیقاً عبارت زیر باشد: Please send a text message so I can repeat it.

اگر هر چهار نتیجه یکسان بود، مسیر کامل شامل تلگرام، توکن، پایتون، تله‌بات و پاسخ‌دهنده‌ها درست کار می‌کند.

۲.۵ اجرای آزمون‌های آفلاین

ربات را با Ctrl+C متوقف کنید؛ سپس اجرا کنید:

```
python -m unittest -v
```

نتیجه مورد انتظار

چهار آزمون فهرست می‌شوند و سپس عبارت OK نمایش داده می‌شود. این بررسی‌ها به توکن نیاز ندارند و به تلگرام متصل نمی‌شوند؛ آن‌ها تابع‌های سازنده پاسخ را بررسی می‌کنند.

هر زمان پیامی را در فایل bot.py تغییر دادید، آزمون متناظر را نیز به‌روز و این دستور را دوباره اجرا کنید.

ایستگاه بررسی

ربات شما به هر چهار بررسی دستی پاسخ درست می‌دهد و چهار آزمون آفلاین موفق هستند.

فصل ۶

رفع مشکل‌های رایج و ادامه مسیر

هدف فصل

مشکل‌های محتمل اجرای نخست را حل کنید و گام بعدی یادگیری را بشناسید.

۱.۶ راهنمای سریع مشکل‌ها

کاری که باید انجام دهید	آنچه می‌بینید
محیط درست را فعال کنید و دستور زیر را اجرا کنید: <code>python -m pip install -r requirements.txt</code>	No module named telebot
متغیر محیطی را در همان ترمینالی تنظیم کنید که دستور <code>python bot.py</code> را اجرا می‌کنید.	BOT_TOKEN is missing
توکن کامل را بدون فاصله دوباره کپی کنید. اگر احتمال فاش شدن آن وجود دارد، ابتدا در بات‌فادر باطلش کنید.	خطای Unauthorized یا خطای توکن
در تلگرام دکمه شروع را بزنید، نام کاربری را بررسی کنید، ترمینال را باز نگه دارید و اتصال اینترنت را بررسی کنید.	برنامه در حال اجراست ولی ربات پاسخ نمی‌دهد
نسخه دیگر ربات را متوقف کنید. فقط یک فرایند دریافت پیام باید از یک توکن استفاده کند.	خطای اجرای هم‌زمان polling

۲.۶ ساخت این کتاب در تک‌استودیو

فایل `main.tex` نسخه مورد نظر را باز کنید، موتور XeLaTeX را انتخاب کنید و دو بار خروجی بگیرید تا فهرست مطالب به‌روز شود. فایل هر فصل را به‌تنهایی کامپایل نکنید. نسخه فارسی به XePersian نیاز دارد و در صورت نصب‌بودن از قلم IRANSansX استفاده می‌کند. قلم‌های B Nazanin و Noto Naskh Arabic نیز به‌عنوان جایگزین تنظیم شده‌اند.

فایل‌های ساخته‌شده مانند PDF, AUX, LOG, TOC و خروجی SyncTeX در گیت نادیده گرفته می‌شوند. آن‌ها را روی رایانه نگه دارید یا برای انتشار، فایل پی‌دی‌اف را به یک انتشار گیت‌هاب پیوست کنید. خروجی‌های تکراری ساخت را در مخزن ذخیره نکنید.

۳.۶ گام‌های کوچک بعدی

پس از درست‌کارکردن این نسخه، هر بار فقط یک تغییر بسازید: دستور about / را اضافه کنید، متن خوشامد را تغییر دهید، دکمه پاسخ بسازید یا ربات را روی خدمتی قرار دهید که پایتون را همیشه روشن نگه می‌دارد. هنگام استقرار نیز توکن را هرگز در کد ننویسید.

نتیجه مورد انتظار

شما یک ربات واقعی تلگرام را ساخته‌اید، فهمیده‌اید و بررسی کرده‌اید. پروژه عمده کوچک است و اکنون پایه‌ای امن برای قابلیت بعدی شماست.

درباره نویسنده

ایده و محتوای این آموزش را Alireza Khajehvandi ساخته است تا تازه‌کارها در زمانی کوتاه به نتیجه‌ای واقعی برسند و هر فایلی را که اجرا می‌کنند، بفهمند.

برای ادامه یادگیری و ارتباط:

- [LinkedIn](#)
- [کانال YouTube](#)
- [کانال تلگرام](#)
- [Instagram](#)
- [GitHub](#)
- [وبسایت شخصی و نمونه‌کارها](#)